05

Part 5 · Reliability & Disaster Recovery

Designing systems that stay up, and recover quickly when something fails.

- Region vs Availability Zone
- High Availability
- Fault Tolerance
- SLA / SLO / SLI
- Disaster Recovery
- RTO vs RPO

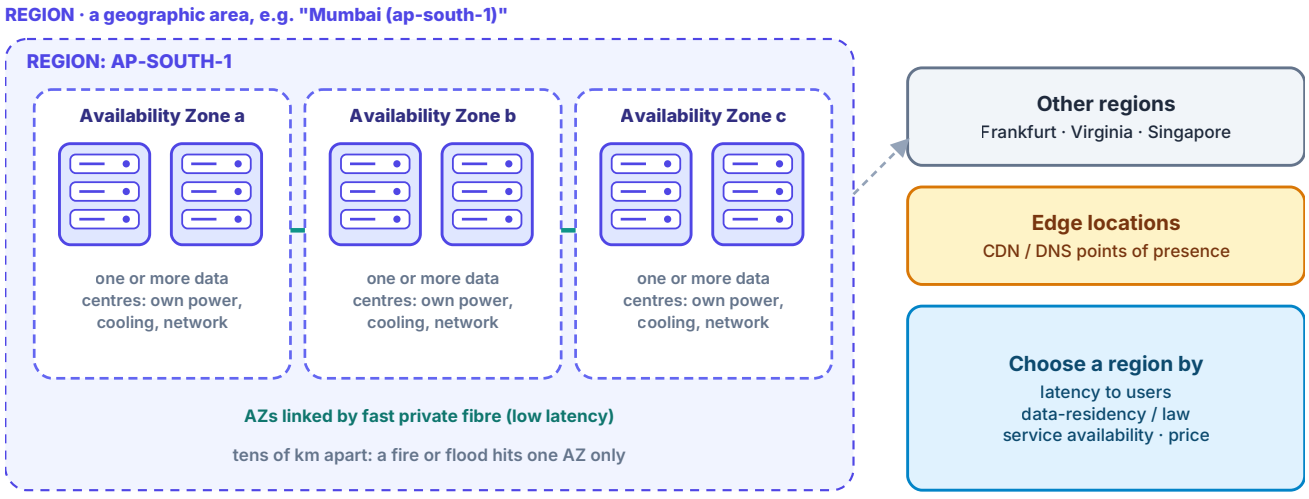
5.1 Region vs Availability Zone

★★★

IN ONE LINE

Regions are geographic areas; Availability Zones are isolated data-centre groups inside a region.

A **Region** is a geographic location containing multiple, isolated data-centre groups called **Availability Zones (AZs)**. AZs are separate buildings with independent power, cooling and networking, close enough for fast links but far enough that one disaster does not take down all.



Global infrastructure = many Regions, each with ≥ 3 AZs (typically), plus a worldwide edge network.

Azure & GCP use the same ideas (Azure: regions + availability zones; GCP: regions + zones).

Figure 5.1 — Regions contain Availability Zones; edge locations sit around them

	Region	Availability Zone
What	A geographic area (e.g. Mumbai, Frankfurt)	An isolated location inside a region
Failure it survives	Natural disaster, regional outage (if multi-region)	Data-centre failure (power, fire, network)
Latency between	Higher (tens to hundreds of ms)	Very low (single-digit ms)
Used for	Disaster recovery, data residency, latency to users	**High availability** inside a region
Data transfer cost	Charged between regions	Small charge between AZs, free inside one

INTERVIEW ANGLE

Multi-AZ = high availability. Multi-region = disaster recovery (and global latency). Most apps need multi-AZ; only the most critical need multi-region because it is costly and complex. Also mention that many services are **regional** (S3, Lambda) while others are **zonal** (EBS volumes, single VMs).

5.2 High Availability

★★★

IN ONE LINE

Keeping a system accessible with minimal downtime by removing single points of failure.

High availability (HA) means designing a system so it stays **accessible with minimal downtime** despite failures. You achieve it by removing **single points of failure** (SPOFs): run redundant copies, in separate failure domains, with automatic detection and failover.

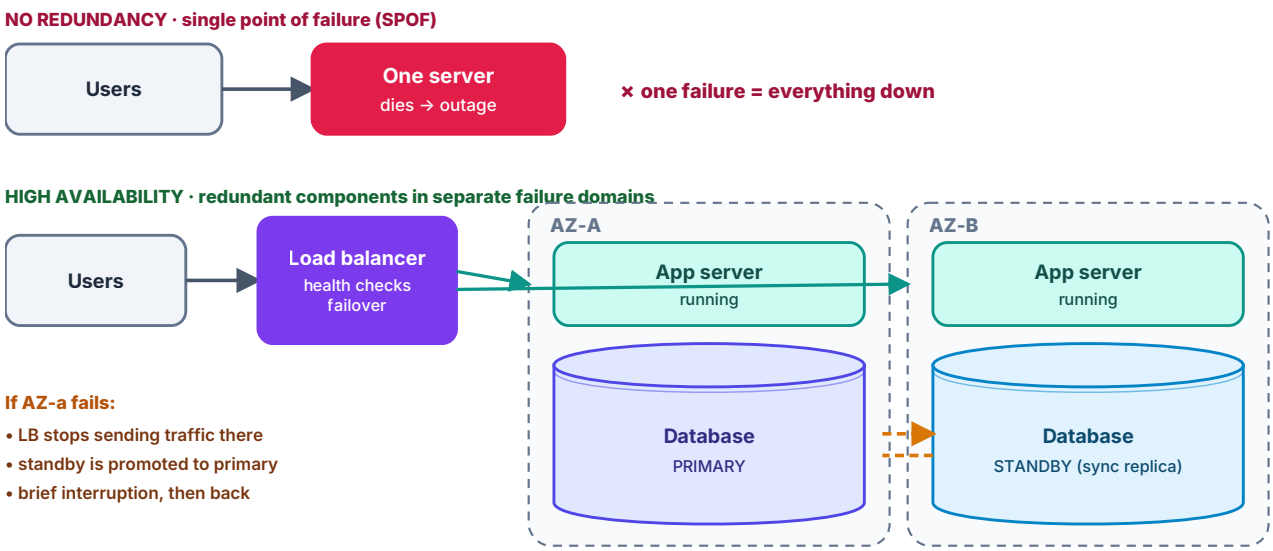


Figure 5.2 — From one server to a multi-AZ, redundant design

HA building blocks

- **Redundancy** at every tier (servers, DBs, load balancers).
- **Multiple AZs**.
- **Health checks** and automatic **failover**.
- **Auto scaling** replaces failed instances.
- **Stateless** app tier; replicated data tier.

Measuring it

Availability % over time: **99.9 %** ≈ 8.8 h downtime/year, **99.99 %** ≈ 53 min. See the table in 3.6 and the SLA topic (5.5).

Active-passive: standby waits. **Active-active**: all nodes serve traffic.

REMEMBER

HA aims for **minimal** downtime (seconds to minutes during failover). It does not promise *zero* interruption — that is fault tolerance (next topic).

INTERVIEW ANGLE

Hunt the SPOF: “What if this one component dies?” applied to the load balancer, the database, the DNS, the single AZ, the one engineer who knows the system. This question structure impresses interviewers.

5.3 Fault Tolerance

★★★

IN ONE LINE

Operating continuously, with no interruption, even when components fail.

Fault tolerance is the ability of a system to **keep operating without any interruption** when a component fails. Instead of "recover quickly" (HA), a fault-tolerant system is built so the failure is **invisible** to users — usually via duplicated components running simultaneously.

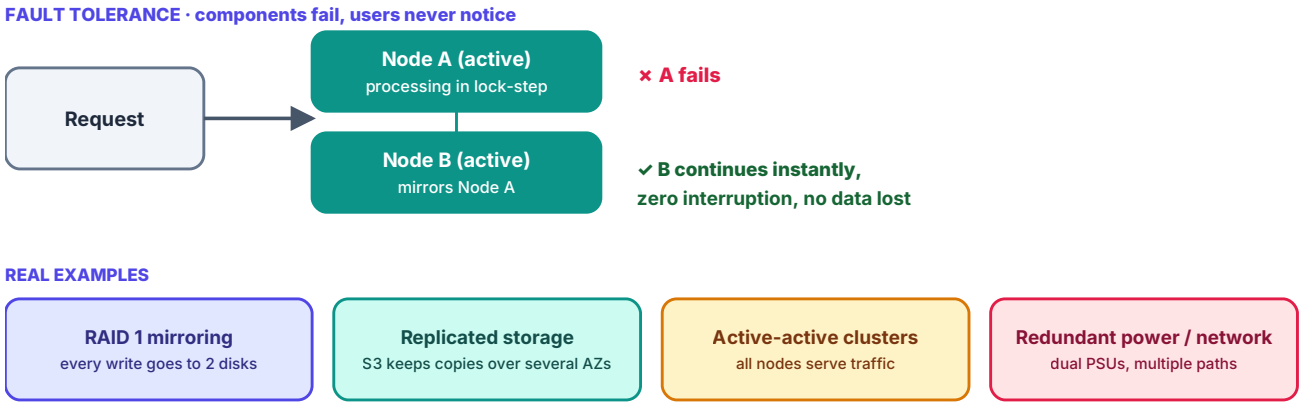


Figure 5.3 — Duplicate, run in parallel, mask the failure

Techniques

- **Redundancy** of hardware and data.
- **Replication** with synchronous writes.
- **Active-active** designs and quorum (majority) systems.
- **Graceful degradation** and retries with back-off.
- **Chaos engineering** (e.g. Netflix Chaos Monkey) to test it.

Trade-offs

- Higher cost (idle-equivalent capacity).
- Added complexity and synchronisation overhead.
- Needed mainly for life-critical or financial systems.

REAL-WORLD ANALOGY

An aircraft with two engines can keep flying if one fails (fault tolerant). A car with a spare tyre can continue after you stop and swap it (highly available: brief interruption, then back).

5.4 High Availability vs Fault Tolerance

★★★

IN ONE LINE

HA minimises downtime; fault tolerance eliminates it.

Both aim to survive failures, but with different promises. **HA = minimise downtime. FT = no downtime at all.**

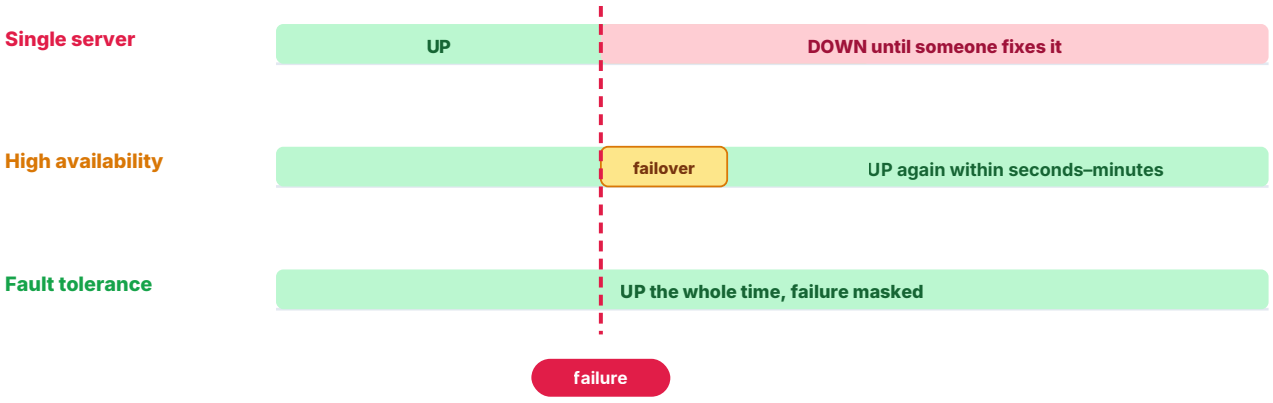


Figure 5.4 — Same failure, three experiences

	High availability	Fault tolerance
Goal	Minimal downtime	Zero downtime
On failure	Detect, fail over; brief interruption possible	Continues seamlessly, nothing noticed
Redundancy	Standby / extra capacity ready	Components run simultaneously, in lock-step
Data loss on failure	Possible with async replication	None
Cost / complexity	Moderate	High
Typical target	99.9–99.99 % uptime	Effectively continuous operation
Examples	Multi-AZ database with automatic failover; ASG behind a load balancer	Mirrored storage, active-active clusters, redundant power

INTERVIEW ANGLE

A great one-liner: **“HA recovers fast; FT never goes down.”** And the practical point: most business applications choose **HA** because it is far cheaper; FT is reserved for systems where even seconds of downtime are unacceptable.

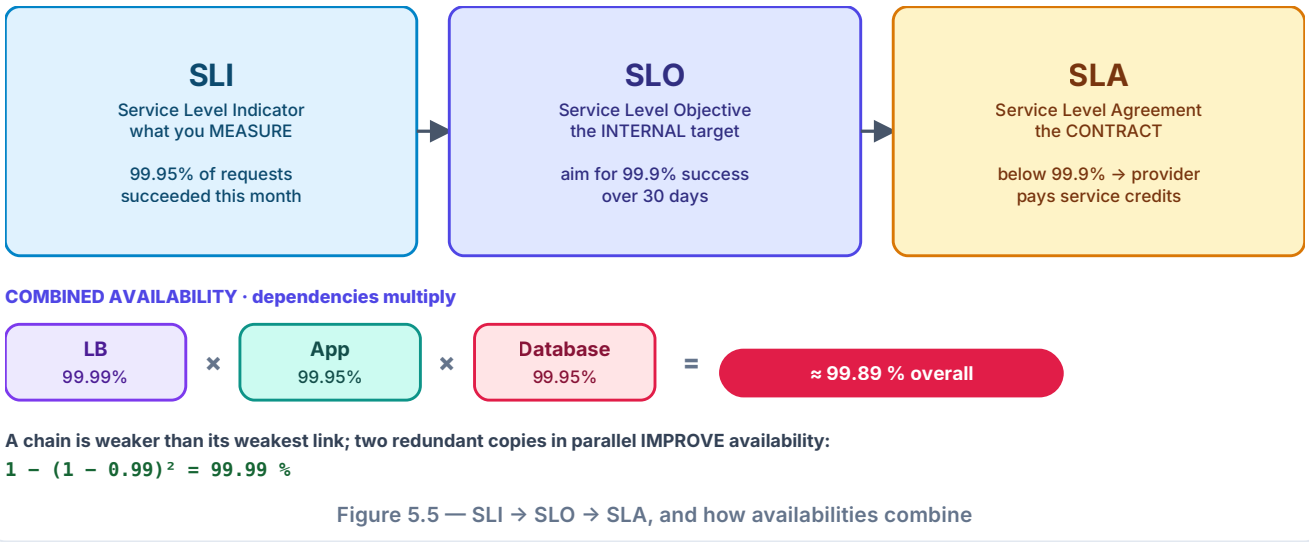
5.5 SLA, SLO & SLI

★★☆

IN ONE LINE

Measuring and promising reliability: indicators, objectives and contractual agreements.

How reliability is promised and measured. **SLI** is the metric, **SLO** is the goal for that metric, **SLA** is the legal contract with consequences if missed. Cloud providers publish SLAs per service (e.g. 99.99 % for a multi-AZ database).



A chain is weaker than its weakest link; two redundant copies in parallel IMPROVE availability:

```
1 − (1 − 0.99)² = 99.99 %
```

Figure 5.5 — SLI → SLO → SLA, and how availabilities combine

Term	Meaning	Example
SLI	A measured indicator of service health	request success rate, latency p95, uptime
SLO	Target value for an SLI (internal)	99.9 % of requests under 300 ms
SLA	External contract, with penalties / credits	“99.95 % monthly uptime or 10 % credit”
Error budget	Allowed unreliability = 100 % − SLO	0.1 % ≈ 43 min per month to spend on changes

COMMON TRAP

A provider’s SLA **only covers their part**, and the remedy is usually service credits — not compensation for your lost business. Your **own** architecture (multi-AZ, retries, backups) decides your real availability.

5.6 Disaster Recovery

★★★

IN ONE LINE

Restoring systems and data after a major outage or loss, using backups and cross-region copies.

Disaster Recovery (DR) is the set of policies and technology for **restoring systems and data after a major failure** — a region outage, ransomware, data corruption, human error. It is part of **business continuity**. Backups (3.7) and multi-region replication are its main tools.



Figure 5.6 — The four classic DR strategies and the cost/speed trade-off

Strategy	How it works	Recovery time	Cost
Backup & restore	Periodic backups copied off-site; rebuild on demand	Hours	$
Pilot light	Minimal always-on core (data replicated); build the rest at failover	Tens of minutes	$$
Warm standby	Scaled-down but functional copy always running	Minutes	$$$
Active-active (multi-site)	Full capacity in 2+ regions at once	Near zero	$$$$

HA vs DR

HA handles *component and AZ* failures automatically inside a region. **DR** handles *region-level or catastrophic* events, often with a manual or semi-automatic decision.

Cloud advantages

Pay for DR capacity only when used, replicate easily across regions, automate with IaC (7.5), and run failover drills cheaply.

INTERVIEW ANGLE

Pick the strategy from the business: **how long can we be down (RTO) and how much data can we lose (RPO)?** Then price the options. Always mention **testing** the plan.

5.7 RTO vs RPO

★★★

IN ONE LINE

RPO: how much data you can lose. RTO: how long you can be down.

The two numbers that define a DR plan. **RPO (Recovery Point Objective)** — the maximum amount of **data** you can afford to lose, measured as time back from the disaster. **RTO (Recovery Time Objective)** — the maximum **downtime** you can tolerate before service must be back.

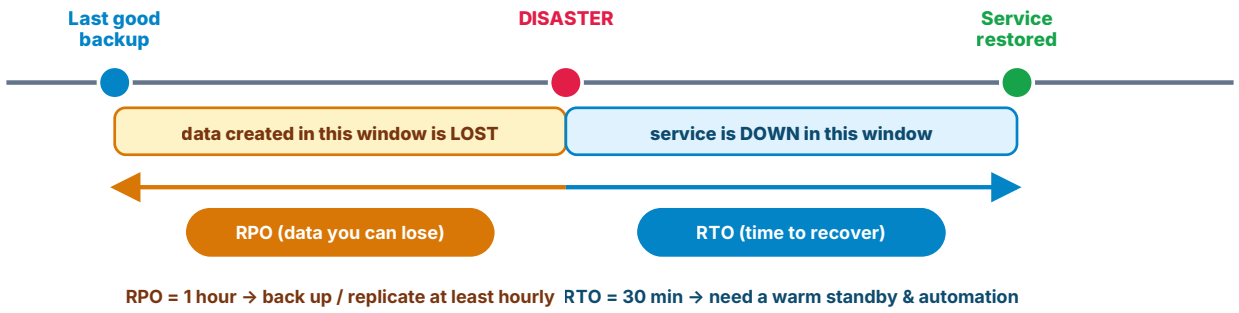


Figure 5.7 — RPO looks back in time; RTO looks forward

	RPO	RTO
Measures	Acceptable **data loss**	Acceptable **downtime**
Direction	Backwards from the disaster	Forwards from the disaster
Question	"How old can the restored data be?"	"How fast must we be running again?"
Drives	Backup / replication frequency	DR strategy and automation level
Smaller means	Replication more frequent (synchronous ~ 0)	Hot / warm standby, automated failover
Example	RPO 15 min → back up every 15 min	RTO 1 hour → restore must finish in 1 hour

REMEMBER

Lower RPO / RTO = higher cost. RPO ≈ 0 needs synchronous replication; RTO ≈ 0 needs active-active. Different systems get different targets: payments need tiny values; an internal wiki does not.

INTERVIEW ANGLE

Trick-question check: "Our RPO is 24 hours and RTO is 5 minutes — what does that mean?" → losing up to a day of data is acceptable, but service must return within minutes (e.g. automated failover to a standby restored from nightly backups).